

Global Farm Size, Version 1, 2000; Augented for FAO SOFA 2025

Zia Mehrabi and Vincent Ricciardi

April 17, 2025

Contents

1	Aim of document	2
2	Reproducibility	2
3	Data	2
3.1	Country boundaries	3
3.2	Country farm size distributions	3
3.3	Global agricultural area	6
3.4	Global field sizes	7
3.4.1	Read in Data	7
3.4.2	Pre-process Data	7
3.4.3	Interpolate	8
3.4.4	Rasterize	9
4	Create common dataframe	9
4.1	Convert to equal area	9
4.2	Set extents	10
4.3	Create stack	10
4.4	Finalize data set	10
5	Match Field Sizes to Farm Sizes	12
5.1	Algorithm	12
5.2	Resampled Results	16
5.3	Sanity Check	16
5.4	ESDA Checks	17
5.5	Final Output	19
	References	21

1 Aim of document

This dataset was originally created for the peer-reviewed publication:

Mehrabi, Z., McDowell, M.J., Ricciardi, V. et al. 2020. The global divide in data-driven farming. *Nature Sustainability*, 4:154–160. <https://doi.org/10.1038/s41893-020-00631-0> [1].

This is an augmented version for the United Nations Food and Agricultural Organization’s (FAO) State of Food and Agriculture Report.

The aim of this document is to create a global map of farm sizes. We do this by merging two currently available sources of information on the distribution of farm sizes across the world: national level data on farm size distributions and subnational data on field size distributions. The resulting map is a dataset of where different sizes of farms are distributed globally. As with the geospatial products on which the map is derived, we caution that this map should not be used for detailed within country-level analysis, but is intended for global assessment studies, where results are typically aggregated at the regional or global level. This document explains the underlying data, pre-processing, and algorithm we developed to create this map. We were driven to create this map as a work that can be built upon and improved as higher resolution and more accurate data become available.

2 Reproducibility

Here we call the **R** package `renv` [2]. This will create a local library on your computer and install a copy of the packages required by this project as they existed on CRAN by the specified version number, and update the **R** session to use these packages. This helps make our analysis fully reproducible on your machine.

Note, the ‘`renv.lock`’ file needs to be in the top level of this project’s directory. This code block needs to only be run when initially setting up your project then can be commented out.

```
> # Uncomment during first run if using renv for install
> # install.packages('renv')
> # renv::init()
```

We also set the seed of the entire document to ensure the same results when randomly sampling.

```
> set.seed(123)
```

Note that the **R** version used here is 4.4.3 (2025-02-28).

For the analysis in this document we will be using the following packages: `SexprR-data.table` `citeR-data.table`.

3 Data

In this analysis, we leverage three key datasets. LINEQ database (forthcoming) (also referred to as Lowder, hereafter, to keep consistent code with the Mehrabi, McDowell, and Ricciardi et al. 2024 paper) contain farm size distributions (in terms of agricultural area and the number of farms) for 100 countries. Lesiv and Fritz et al. 2019 [3] (Lesiv, hereafter) contains a crowd-sourced point data of categorical field size classes (ranging from very small to large fields). We use Lesiv’s ordinal field size classes to spatially disaggregate the LINEQ farm size classes, using known agricultural area from GAEZ 2025 (also referred to as Ramankutty, hereafter, to keep consistent code with the Mehrabi, McDowell, and Ricciardi et al. 2024 paper). Links and access dates to these input data and other ancillary data sets used in our analysis as highlighted below:

1. Global field size data from Lesiv, retrieved from <http://pure.iiasa.ac.at/id/eprint/15526/> on October 12th, 2018.
2. Country farm size distributions from LINEQ (please contact author)
3. Global cropland and pastureland from GAEZ, retrieved from <https://console.cloud.google.com/storage/browser/fao-gismgr-gaez-v5-data/DATA/GAEZ-V5/MAPSET/LR-LCC> on April 15th 2025.

4. Country boundaries, retrieved internally in R through [4].

3.1 Country boundaries

First we make a raster of the world country data from the `rworldmap` package.

```
> world <- rworldmap::getMap(resolution = 'low')
> lookup <- as.data.frame(cbind(as.character(
+   world@data[, 'ISO3']),
+   world@data[, 'ADMIN'],
+   as.character(world@data[, 'ADMIN'])))
> raster.world <- raster(res = c(0.0833282, 0.0833282))
> extent(raster.world) <- extent(world)
> world.raster <- rasterize(as(world, 'SpatialPolygons'),
+   raster.world,
+   field = world@data[, 'ADMIN'],
+   fun = 'first')
> world.rast <- writeRaster(world.raster,
+   'data/tmp/worldrast.tif',
+   format = 'GTiff',
+   overwrite = TRUE)
> world <- raster('data/tmp/worldrast.tif')
```

3.2 Country farm size distributions

Lowder's data contains two variables at the national level, the amount of agricultural area per farm size class, and the number of farms per farm size class. We are interested in the amount of agricultural area per farm size class for our analysis. This data is distributed as the World Census of Agriculture's (WCA) farm size classes, which are: 0-1 ha, 1-2 ha, 2-5 ha, 5-10 ha, 10-20 ha, 20-50 ha, 50-100 ha, 100-200 ha, 200-500 ha, 500-1000 ha, 1000 ha. This data is read in here and columns relabeled.

```
> # Load LINEQ's distribution dataset
> lwd <- read.csv('data/lineq/lineq_fs_distributions.csv',
+   header = T,
+   na.strings = c('', 'NA'))
> lwd$Income <- lwd$Region <- NULL
> #####
> # Clean up country names, since lineq database uses the country name from the year of the data (e.g.,
> world_tmp <- getMap(resolution = 'low')
> countries <- data.frame(names = sort(unique(as.character(world_tmp@data[, 'ADMIN']))))
> lwdNames <- data.frame(names = unique(lwd$Country))
> lwdNames$names <- str_trim(lwdNames$names)
> lwdNames$indata <- lwdNames$names %in% unique(countries$name)
> # List of names in LINEQ that do not match dataset's names
> # lwdNames[which(lwdNames$indata == FALSE), ]$names
>
> changeFrom <- c(
+   "Bahamas",
+   "Ceylon",
+   "China, People's Republic of",
+   "Congo",
+   "Congo - Brazzaville",
+   "Congo - Kinshasa",
+   "Côte d'Ivoire",
+   "Czechia",
```

```

+ "Czechoslovakia",
+ "Eswatini",
+ "Guadeloupe",
+ "Guinea-Bissau",
+ "Korea, Rep. of",
+ "Korea, Republic of",
+ "Martinique",
+ "Micronesia, Federated States of",
+ "Myanmar (Burma)",
+ "North Macedonia",
+ "Pacific islands (trust territory)",
+ "Palestine",
+ "Republic of Korea",
+ "Réunion",
+ "Russian Federation",
+ "Serbia",
+ "South West Africa",
+ "Southern Rhodesia (European holdings)",
+ "St. Lucia",
+ "St. Vincent & Grenadines",
+ "Tanganyika (commercial holdings only)",
+ "Tanzania",
+ "Timor-Leste",
+ "Trinidad & Tobago",
+ "U.S. Virgin Islands",
+ "United States",
+ "Viet Nam",
+ "Virgin Islands (U.K.)",
+ "Yugoslavia",
+ "Zambia (European holdings)"
+ )
> changeTo <- c(
+ "The Bahamas",
+ "Sri Lanka",
+ "China",
+ "Republic of the Congo",
+ "Republic of the Congo",
+ "Democratic Republic of the Congo",
+ "Ivory Coast",
+ "Czech Republic",
+ "Czech Republic",
+ "Swaziland",
+ "Guadeloupe", # Keep as is but now matches
+ "Guinea Bissau",
+ "South Korea",
+ "South Korea",
+ "Martinique", # Keep as is but now matches
+ "Federated States of Micronesia",
+ "Myanmar",
+ "Macedonia",
+ "Pacific Islands",
+ "West Bank",
+ "South Korea",
+ "Reunion", # Removed accent

```

```

+ "Russia",
+ "Republic of Serbia",
+ "Namibia",
+ "Zimbabwe",
+ "Saint Lucia",
+ "Saint Vincent and the Grenadines",
+ "United Republic of Tanzania",
+ "United Republic of Tanzania",
+ "East Timor",
+ "Trinidad and Tobago",
+ "United States Virgin Islands",
+ "United States of America",
+ "Vietnam",
+ "British Virgin Islands",
+ "Republic of Serbia",
+ "Zambia"
+ )
> require(plyr)
> lwd$Country <- mapvalues(str_trim(lwd$Country),
+                          from = changeFrom,
+                          to   = changeTo)
> # Find which original names map to the same transformed name
> # original_names <- unique(lwd$Country)
> # transformed_names <- mapvalues(original_names, from = changeFrom, to = changeTo)
> # duplicated_mappings <- transformed_names[duplicated(transformed_names)]
> #
> # # For each duplicated transformed name, find all original names that map to it
> # for (dup_name in unique(duplicated_mappings)) {
> #   cat("Multiple names map to:", dup_name, "\n")
> #   cat("Original names:", paste(original_names[transformed_names == dup_name], collapse=", "), "\n\n")
> # }
>
> detach('package:plyr')
> #####
>
> # For lineq data, ensure most recent year
> lwd <- lwd %>%
+   group_by(Country, Holdings..agricultural.area) %>%
+   dplyr::arrange(desc(Census.Year)) %>%
+   slice(1)
> # Subset only agricultural area
> lwd <- lwd[which(lwd$Holdings..agricultural.area != 'Holdings'), ]
> lwd <- lwd[, c(1,5:15)]
> names(lwd) <- c('country', '0_1', '1_2', '2_5',
+                '5_10', '10_20', '20_50', '50_100', '100_200',
+                '200_500', '500_1000', '1000_5000')
> # Ensure variable type is numeric
> for (i in names(lwd)[2:length(names(lwd))]) {
+   lwd[[i]] <- as.numeric(lwd[[i]])
+ }

```

Next we need to calculate the cumulative sum per country across LINEQ's farm size classes. To do this we first convert the data from wide to long format, and set classes for countries without data to zero area.

```

> lwd <- melt(lwd, id.vars = c('country'))

```

```
> lwd[is.na(lwd)] <- 0
> # Save an instance of this lwd dataset for later sanity checks
> lwd_orig <- lwd
```

Then we calculate the proportional agricultural area for each farm size class for each country. We note that LINEQ contains some countries without farm size distributions and some countries do not have a distribution for each farm size class, and so we remove those cases here.

```
> lwd <- lwd %>%
+   group_by(country) %>%
+   mutate(total = sum(value),
+           perc = value / total) %>%
+   filter(value > 0) %>%
+   select(country, variable, perc)
```

Finally, cast dataframe back into wide form.

```
> lwd <- dcast(lwd, country ~ variable)
> write.csv(lwd, 'data/tmp/lwd.csv')
```

3.3 Global agricultural area

We use GAEZ's 1 minute resolution cropland map (crop, hereafter) as our reference layer of the distribution of agricultural land at the subnational level. We read in these files here and aggregate to the 5 minute (10km resolution) by computing the weighted mean of cropland areas while excluding areas with zero cropland. This makes our aggregated values representative of actual cropland areas rather than the entire grid cell (ag, hereafter).

```
> require(terra)
> # read data
> cropland_1km <- 'data/gaez_v5/GAEZ_1km/GAEZ-V5.LR-LCC.LC02.tif'
> cropland_1km <- rast(cropland_1km)
> # reprojec tto equal area
> if (crs(cropland_1km) != "+proj=longlat +datum=WGS84 +no_defs") {
+   cropland_1km <- project(cropland_1km, "+proj=longlat +datum=WGS84 +no_defs")
+ }

|-----|-----|-----|-----|
=====

> # mask cells with 0% cropland
> cropland_1km[cropland_1km == 0] <- NA

|-----|-----|-----|-----|
=====

|-----|-----|-----|-----|
=====

> # aggregate to the 10km resolution
> cropland_10km <- aggregate(cropland_1km, fact=10, fun="mean", na.rm = TRUE)

|-----|-----|-----|-----|
=====

> # Convert values to a 0 to 1 scale
> cropland_10km <- cropland_10km / 100
> # Write raster
> writeRaster(cropland_10km, "data/gaez_v5/GAEZ_1km/GAEZ-V5.LR-LCC.LC02_10km.tif", overwrite = TRUE)
```

```

> # Read in cropland (keep pasture data as a placeholder
> # in case team decides to use in future versions)
> crop <- 'data/gaez_v5/GAEZ_1km/GAEZ-V5.LR-LCC.LC02_10km.tif'
> pasture <- paste0('data/Ramankutty_2008_cropland/',
+                  'CroplandPastureArea2000_Geotiff/',
+                  'Pasture2000_area.tif')
> crop <- raster(crop)
> pasture <- raster(pasture)
> # Reset NA to 0 for later computational ease
> crop_tmp <- crop
> pasture_tmp <- pasture
> crop_tmp[is.na(crop_tmp)] <- 0
> pasture_tmp[is.na(pasture_tmp)] <- 0
> origin(crop_tmp) <- origin(pasture_tmp) <- c(0.0, 0.0)
> ag <- mosaic(crop_tmp, pasture_tmp, fun = sum)
> ag[ag[] == 0] <- NA
> # use only cropland
> ag <- crop_tmp

```

3.4 Global field sizes

Here we interpolate the crowd-sourced field size data point data onto GAEZ's agricultural area data. While the original sampling frame for this field size was on a unionized cropland map, we extend the distribution here in an attempt to better match and account for the inclusion of non-cropland in LINEQ's farm size data. We use *kk*-nearest neighbor (*kkNN*), a weighted variant of *kNN* that we use to try to account as best as possible for Lesiv's sparsely sampled points in some locations, e.g. Africa).

3.4.1 Read in Data

First, we read in the estimated dominant field sizes point data and subset to not include NA values or values coded as having no fields.

```

> fs <- read.csv("data/Fritz_2019/Global Field Sizes/estimated_dominant_field_sizes.csv")

```

3.4.2 Pre-process Data

We recode Lesiv's data according to their coding as detailed below. And then convert this data into a spatial object.

- 3502 - Very large fields with an area of greater than 100 ha
- 3503 - Large fields with an area between 16 ha and 100 ha
- 3504 - Medium fields with an area between 2.56 ha and 16 ha
- 3505 - Small fields with an area between 0.64 ha and 2.56 ha
- 3506 - Very small fields with an area less than 0.64 ha
- 3507 - no fields
- NA - skipped

```

> fs[which(fs$field_size == 3507), 5] <- NA
> fs_clean <- fs %>%
+   na.omit() %>%
+   dplyr::mutate(field_size_fac = as.factor(field_size)) %>%
+   dplyr::mutate(fs_verbatim_fac = recode(field_size_fac,
+                                          "3502" = "very large",

```

```

+             "3503" = "large",
+             "3504" = "medium",
+             "3505" = "small",
+             "3506" = "very small")))%>%
+   dplyr::select(-rowid,-sampelid) %>%
+   st_as_sf(coords = c("x", "y"),
+            crs = "+proj=longlat +ellps=WGS84")

```

3.4.3 Interpolate

We make a target raster grid to interpolate onto using the agricultural area raster. We use a very simple method of interpolation based on the latitude and longitude.

```

> ag_GRO <- ag
> ag_GRO[] <- ifelse(ag_GRO[] > 0, 1, NA)
> target_pts <- rasterToPoints(ag, spatial = TRUE)
> target_pts_agGRO <- rasterToPoints(ag_GRO, spatial = TRUE)

```

We then train the kkNN model using different types of kernels and number of points used in the classification procedure. We use 10 fold cross validation to identify the best parameters to use in the final model (noting that CV does not approximate the test error as would be estimated using an independent hold-out set).

```

> fs_train <- data.frame(field_size = fs_clean$fs_verbatim_fac,
+                        x = st_coordinates(fs_clean)[,1],
+                        y = st_coordinates(fs_clean)[,2])
> # fs_train %>% group_by(field_size) %>% summarise(n()) # decently balanced
>
> fs_interp_train <- kknn::train.kknn(field_size ~.,
+                                     data = fs_train,
+                                     ks = seq(5, 51, 5),
+                                     distance = 2,
+                                     kernel = c("gaussian",
+                                                "triangular",
+                                                "rectangular",
+                                                "epanechnikov",
+                                                "optimal"
+                                                ),
+                                     kcv = 10)

```

Next we summarize and plot the results of the model fitting. We also plot the error from the different models below. We note the minimal misclassification error is 0.43. Crudely, with a balanced dataset of 5 classes you would expect a null misclassification error of 0.8, and so under this special case the best model halves uncertainty in predictions from a random classifier. Future versions improvements to this model may be made through using haversine distance, or identifying ancillary features that do a better job of predicting field size classes. Similarly, using an ordinal response, instead of the nominal response defaulted here, may improve predictions also.

Note, that there is class confusion between the very small and small and very large and large. The overall accuracy is 0.62 but if we collapse classes the overall accuracy is 0.743. The medium field size has the largest misclassification. If we collapse the medium into the larger field sizes then the accuracy is 0.853; conversely if we collapse the medium into the smaller field sizes then the accuracy is 0.845, which suggests there's slightly more confusion with medium and larger field sizes.

We use the best fitting model parameters to predict the field size classes across the agricultural area grid, and then extract the fitted values and the associated probabilities.

```

> fs_interp_final <- kknn::kknn(field_size ~ .,
+                               train = fs_train,

```



```

+                 test = target_pts_agGRO,
+                 kernel = fs_interp_train$best.parameters$kernel,
+                 k = fs_interp_train$best.parameters$k)
> target_pts_agGRO_res <- target_pts_agGRO %>%
+   # extract the interpolated class at each
+   # grid cell with the kkn::fitted function
+   as.data.frame() %>%
+   # only retain the probability of the
+   # interpolated size class, discard the others
+   mutate(field_size_fit = fitted(fs_interp_final),
+          prob = apply(fs_interp_final$prob, 1, function(input) max(input)))

```

3.4.4 Rasterize

Here we convert results back to a raster and save the results of the fitted classes.

```

> target_pts_agGRO_res_spdf <- target_pts_agGRO_res
> coordinates(target_pts_agGRO_res_spdf) <- ~ x + y
> fs_interp_final_ras <- rasterize(target_pts_agGRO_res_spdf,
+                                 ag,
+                                 as.integer(
+                                   target_pts_agGRO_res_spdf$field_size_fit),
+                                 progress = "text")
> fs_interp_final_ras_prob <- rasterize(target_pts_agGRO_res_spdf,
+                                       ag,
+                                       target_pts_agGRO_res_spdf$prob,
+                                       progress = "text")
> lesiv <- fs_interp_final_ras

```

4 Create common dataframe

To run our algorithm to create a field size map we need to ensure all rasterized datasets (i.e., field size, crop, pasture, ag, and country boundaries) are all in the same resolution, have the same coordinate reference system (crs), and have the same spatial extent, which we do in this section.

4.1 Convert to equal area

Here we set each dataset to have an equal area (Eckert IV) at a 8.4 km x 8.4 km resolution. To interpolate, we use nearest neighbor for the categorical data and bilinear for the continuous data.

```

> rast.list.ngb <- list(lesiv, world)
> # rast.list.bil <- list(crop, pasture, ag)
> rast.list.bil <- list(ag)
> resValue <- 8439
> rast.list.ngb.eq <- lapply(rast.list.ngb, function(x)
+   projectRaster(
+     x,
+     res = c(resValue, resValue),
+     crs = '+proj=eck4 +datum=WGS84 +ellps=WGS84 +towgs84=0,0,0',
+     method = 'ngb',
+     over = T))
> rast.list.bil.eq <- lapply(rast.list.bil, function(x)
+   projectRaster(
+     x,
+     res = c(resValue, resValue),

```

```

+     crs      = '+proj=eck4 +datum=WGS84 +ellps=WGS84 +towgs84=0,0,0',
+     method = 'bilinear',
+     over     = TRUE))
> rast.list <- c(rast.list.ngb.eq, rast.list.bil.eq) # Combine data
> # names(rast.list) <- c('lesiv', 'world', 'crop', 'pasture', 'ag')
> names(rast.list) <- c('lesiv', 'world', 'ag')

```

4.2 Set extents

Next, we need to make sure that all the rasters have the same spatial extents. The country boundaries include Antarctica, while the cropland/pastureland/ag and field size data do not (e.g., crop ymin is -8501789, while the country boundaries ymin is -8503388). To ensure all the datasets match we need to expand the cropland/pastureland/ag and Lesiv extents to match the country boundaries extent. This means extending the cropland/pastureland/ag and Lesiv data to also include the country boundary grid cells; we do this by adding rows of NA grid cells.

```

> ex1 <- extent(rast.list$world)
> # Extend
> rast.list.ex <- lapply(rast.list,
+                        function(x) extend(x, ex1))
> # Crop
> rast.list.c <- lapply(rast.list.ex,
+                      function(x) crop(x, ex1))
> # Force equal extent
> rast.list.et <- lapply(rast.list.c,
+                      function(x) setExtent(x, ex1,
+                                             keepres = TRUE))

```

4.3 Create stack

Now that all the layers' resolution, crs, and spatial extent are equal, we can create a raster stack and proceed with the analysis.

```

> rast.all <- stack(rast.list.et)

```

4.4 Finalize data set

We now convert the data in the raster stack into a dataframe, which we will use for later analysis.

```

> df <- raster::extract(rast.all, 1:ncell(rast.all), df = T)
> df <- as.data.frame(df)
> df$ID <- 1:nrow(df)

```

Set any crop, pasture value under 0.01 to NA, to account for sparse agricultural areas.

```

> # df$crop <- ifelse(df$crop >= 0.01, df$crop, NA)
> # df$pasture <- ifelse(df$pasture >= 0.01, df$pasture, NA)
> df$ag <- ifelse(df$ag >= 0.01, df$ag, NA)

```

Next we add back in the country names (presently they are only country ISO3 IDs), then subset the data to only contain countries in the Lowder dataset (this speeds up processing).

```

> world <- getMap(resolution = 'low')
> lookup <- as.data.frame(
+   cbind(
+     as.character(world@data[, 'ISO3']),
+     world@data[, 'ADMIN'],
+     as.character(world@data[, 'ADMIN'])))

```

```
> matched <- lookup[match(df$world, lookup$V2), ]  
> df$ISO3 <- matched[[1]]  
> df$country <- matched[[3]]
```

Subset the dataset to only the countries containing farm size distributions.

```
> lwdNames <- unique(lwd$country)  
> df_lwd <- df[which(df$country %in% lwdNames), ]  
> df_not_lwd <- df[which(!df$country %in% lwdNames), ]
```

5 Match Field Sizes to Farm Sizes

5.1 Algorithm

Here are the steps we used to create the farm size map, with uncertainty at the level of the distribution of farm to field size matching. The algorithm makes the assumption that smaller field sizes are owned by smaller farms (although we note that the uncertainty in this assumption grows with farm size, as while smaller farms can not own large fields, large farms can own many small fields).

1. Group pixels by country
2. Assign pseudo-random number to pixels
3. Reorder the pixels in a nested fashion: first by interpolated field size classes, and then by the pseudo-random numbers
4. Compute pixel wise agricultural area from GAEZ
5. Compute cumulative agricultural area from GAEZ
6. Divide cumulative sum of agricultural area by total area in each country to get cumulative proportional sums of agricultural area from GAEZ
7. Assign pixel fields sizes to farm size classes by matching the cumulative proportional agricultural area from Lowder to the cumulative proportional agricultural area for interpolated Lesiv product where pixels are ordered from smallest to largest fields
8. For countries not in LINEQ, hard code farm size based on field sizes.
9. Convert result to a raster
10. Save raster
11. Iterate steps 1 to 10 100x

Steps 1 to 6 are written below.

```
> cumsumSkipNA <- function(x) {  
+   FUNC <- match.fun('cumsum')  
+   x[!is.na(x)] <- FUNC(x[!is.na(x)])  
+   return(x)  
+ }  
> df_lwd$lesiv <- factor(df_lwd$lesiv)  
> levels(df_lwd$lesiv) <- c('5', '4', '3', '2', '1')  
> df_lwd$lesiv <- as.integer(as.character(df_lwd$lesiv))  
> steps_1_6 <- function(df_lwd) {  
+  
+   dat <- df_lwd %>%  
+     mutate(pseudorandom = sample(1:n(), n(), replace = F)) %>%  
+     arrange(country, lesiv, pseudorandom) %>%  
+     group_by(country) %>%  
+     mutate(#crop = crop * resValue^2 * 0.0001,  
+           # pasture = pasture * resValue^2 * 0.0001,  
+           # ag = crop + pasture,  
+           ag_area = ag * resValue^2 * 0.0001,  
+           cumsum_area = cumsumSkipNA(ag_area),  
+           prop_sums = ifelse(  
+             is.na(cumsum_area),  
+             NA,  
+             cumsum_area / max(cumsum_area, na.rm = T)  
+           )  
+ }
```

```

+   )
+
+   return(dat)
+ }

```

Step 7 is written below.

```

> cumsumSkipNA <- function(x) {
+   FUNC <- match.fun('cumsum')
+   x[!is.na(x)] <- FUNC(x[!is.na(x)])
+   return(x)
+ }
> step_7 <- function(dat) {
+
+   tmp <- lwd[, 2:length(lwd)]
+   tmp <- t(tmp)
+   tmp <- apply(tmp, FUN = function(x) cumsumSkipNA(x), MARGIN = 2)
+   tmp <- t(tmp)
+   tmp <- as.data.frame(tmp)
+   tmp$country <- lwd$country
+
+   dat <- merge(dat, tmp, by = 'country')
+
+   for (x in unique(dat$country)) {
+
+     tmp <- dat[which(dat$country == x), ]
+
+     tmp$farm <- ifelse(tmp$prop_sums <= tmp$`0_1`
+       & !is.na(tmp$`0_1`),
+       '0_1',
+       ifelse(tmp$prop_sums <= tmp$`1_2`
+       & !is.na(tmp$`1_2`),
+       '1_2',
+       ifelse(tmp$prop_sums <= tmp$`2_5`
+       & !is.na(tmp$`2_5`),
+       '2_5',
+       ifelse(tmp$prop_sums <= tmp$`5_10`
+       & !is.na(tmp$`5_10`),
+       '5_10',
+       ifelse(tmp$prop_sums <= tmp$`10_20`
+       & !is.na(tmp$`10_20`),
+       '10_20',
+       ifelse(tmp$prop_sums <= tmp$`20_50`
+       & !is.na(tmp$`20_50`),
+       '20_50',
+       ifelse(tmp$prop_sums <= tmp$`50_100`
+       & !is.na(tmp$`50_100`),
+       '50_100',
+       ifelse(tmp$prop_sums <= tmp$`100_200`
+       & !is.na(tmp$`100_200`),
+       '100_200',
+       ifelse(tmp$prop_sums <= tmp$`200_500`
+       & !is.na(tmp$`200_500`),
+       '200_500',
+       ifelse(tmp$prop_sums <= tmp$`500_1000`

```

```

+             & !is.na(tmp$`500_1000`),
+             '500_1000',
+             ifelse(tmp$prop_sums <= tmp$`1000_5000`
+                 & !is.na(tmp$`1000_5000`),
+                 '1000_5000',
+                 '1000_5000'
+             )))))))
+
+     if (x == as.character(unique(dat$country)[1])) {
+       out <- tmp
+     } else {
+       out <- rbind(out, tmp)
+     }
+   }
+
+   out <- out[,c(2,length(out))]
+
+   dat <- merge(df, out,
+               by = 'ID',
+               all.x = T,
+               all.y = T)
+
+   return(dat)
+ }

```

Step 8 is written below. For countries not in Lowder, we hard code the farm size matched to that field size class. We take the upper end of the given range and match it to Lesiv's codes (e.g., code 3502 becomes 200 ha)

1 - 3502 - Very large fields with an area of greater than 100 ha - 100-200 ha
 2 - 3503 - Large fields with an area between 16 ha and 100 ha - 50-100 ha
 3 - 3504 - Medium fields with an area between 2.56 ha and 16 ha - 5-10 ha
 4 - 3505 - Small fields with an area between 0.64 ha and 2.56 ha - 1-2 ha
 5 - 3506 - Very small fields with an area less than 0.64 ha - 0-1 ha
 NA - 3507 - no fields;

```

> step_8 <- function(dat) {
+
+   field_2_farm <- data.frame(
+     lesiv = c(1,2,3,4,5),
+     # lesiv = c(5,4,3,2,1),
+     farm_avg = c('100_200',
+                 '50_100',
+                 '5_10',
+                 '1_2',
+                 '0_1'))
+
+   dat <- merge(dat, field_2_farm, by = 'lesiv', all.x = T)
+
+   # Make sure that any inf are NA
+   dat2 <- dat %>% rationalize()
+
+ }

```

```

+ # Make sure farm and farm_avg cats are same type
+ dat$farm <- as.character(dat$farm)
+ dat$farm_avg <- as.character(dat$farm_avg)
+
+ dat$farm_final <- ifelse(dat$ag >= 0.01 & !is.na(dat$farm),
+                           dat$farm,
+                           ifelse(dat$ag >= 0.01 & !is.na(dat$lesiv),
+                                   dat$farm_avg, NA))
+ return(dat)
+ }

```

Step 9: Generate farm size raster

```

> step_9 <- function(dat, x = 'farm_final') {
+ # Places farm size values into a raster
+ # Params:
+ #   df
+ # Return:
+ #   raster object
+
+ # Extract raster values from data
+ if (x == 'farm_final') {
+   dat$rasterValues <- as.numeric(
+     str_split_fixed(dat[[x]], '_', 2)[, 2])
+ } else {
+   dat$rasterValues <- dat[[x]]
+ }
+
+ # Clip to cropland area
+ # dat$rasterValues <- ifelse(is.na(dat$crop), NA, dat$rasterValues)
+ dat$rasterValues <- ifelse(is.na(dat$ag), NA, dat$rasterValues)
+
+ # Sort by ID for consistency
+ dat <- dat[order(dat$ID), ]
+
+ # Get the base raster
+ rast_final <- rast.all@layers[[1]]
+
+ # Create a vector of NAs the same length as the number of cells in the raster
+ all_values <- rep(NA, ncell(rast_final))
+
+ # Filter for valid data points
+ valid_idx <- !is.na(dat$ID) & !is.na(dat$rasterValues) & dat$ID <= ncell(rast_final)
+
+ # Set values only at the specified valid indices
+ if(sum(valid_idx) > 0) {
+   all_values[dat$ID[valid_idx]] <- dat$rasterValues[valid_idx]
+ }
+
+ # Set all values at once
+ rast_final <- setValues(rast_final, all_values)
+
+ return(rast_final)
+ }

```

Step 10: Write raster

```

> step_10 <- function(rast_final, i) {
+   # Writes raster
+   # Params:
+   #   rast_final
+   #   i: random number
+   # Returns
+   #   Writes raster to file
+
+   writeRaster(rast_final,
+               filename = paste0(
+                 # 'raster_out_forCleanRunTest/farmSize_',
+                 'raster_out_forPublication/farmSize_agarea_',
+                 format(Sys.time(), "%Y%m%d"), '_', i, '.tif'),
+               format = 'GTiff',
+               overwrite = TRUE)
+ }

```

5.2 Resampled Results

Step 11: Repeat steps 1-10 100x. Note, 100 runs is illustrative. For more stable results increase to over 1k.

```

> # i = 1
> for (i in 1:100) { # use for full run
+   # for (i in 1:2) { # use for test run
+     dat1 <- steps_1_6(df_lwd)
+     dat2 <- step_7(dat1)
+     dat3 <- step_8(dat2)
+     rast <- step_9(dat3)
+     step_10(rast, i)
+   }

```

5.3 Sanity Check

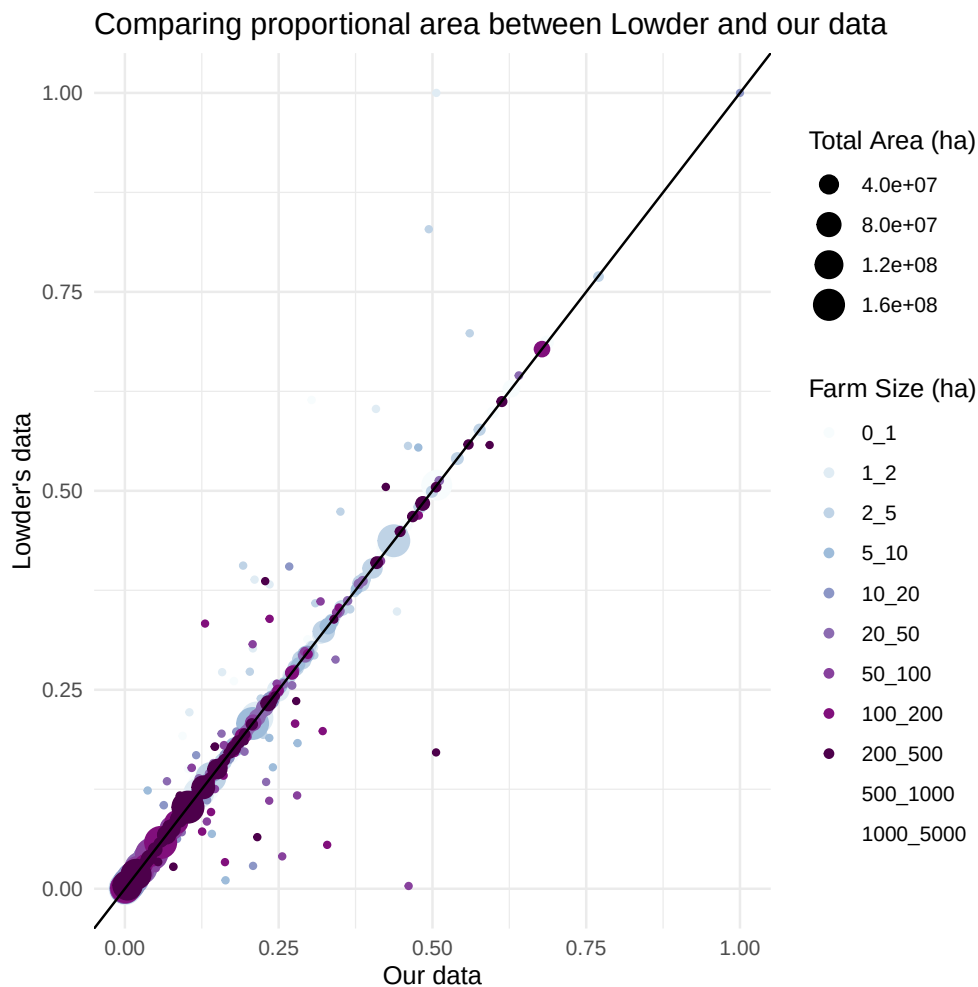
We sanity check that the proportional farm size distributions per country equals Lowder's. We use one run as an example. As we can see the data is very close to the one-to-one line (but not exactly as would be expected based on small differences in row matching in our algorithm). Since the sampling scheme randomly shuffles the data, this plot will be slightly different for each sample.

```

# A tibble: 25 × 5
  country          variable value.xp value.yp diff
  <chr>          <fct>      <dbl>   <dbl> <dbl>
1 Seychelles    1000_5000    0.494     0    0.494
2 Singapore      50_100      0.461   0.00347 0.458
3 American Samoa 200_500      0.506   0.171    0.335
4 Saint Vincent and the Grenadines 100_200    0.329   0.0551 0.274
5 Guam          1000_5000    0.231     0    0.231
6 Bahrain        50_100      0.256   0.0407 0.215
7 Saint Kitts and Nevis 1000_5000    0.200     0    0.200
8 Antigua and Barbuda 10_20       0.208   0.0287 0.180
9 Barbados       50_100      0.280   0.117    0.163
10 Saint Kitts and Nevis 5_10        0.164   0.0106 0.153
11 Grenada       200_500      0.215   0.0648 0.150
12 Saint Lucia   100_200      0.163   0.0334 0.129
13 United States Virgin Islands 50_100      0.235   0.111    0.124

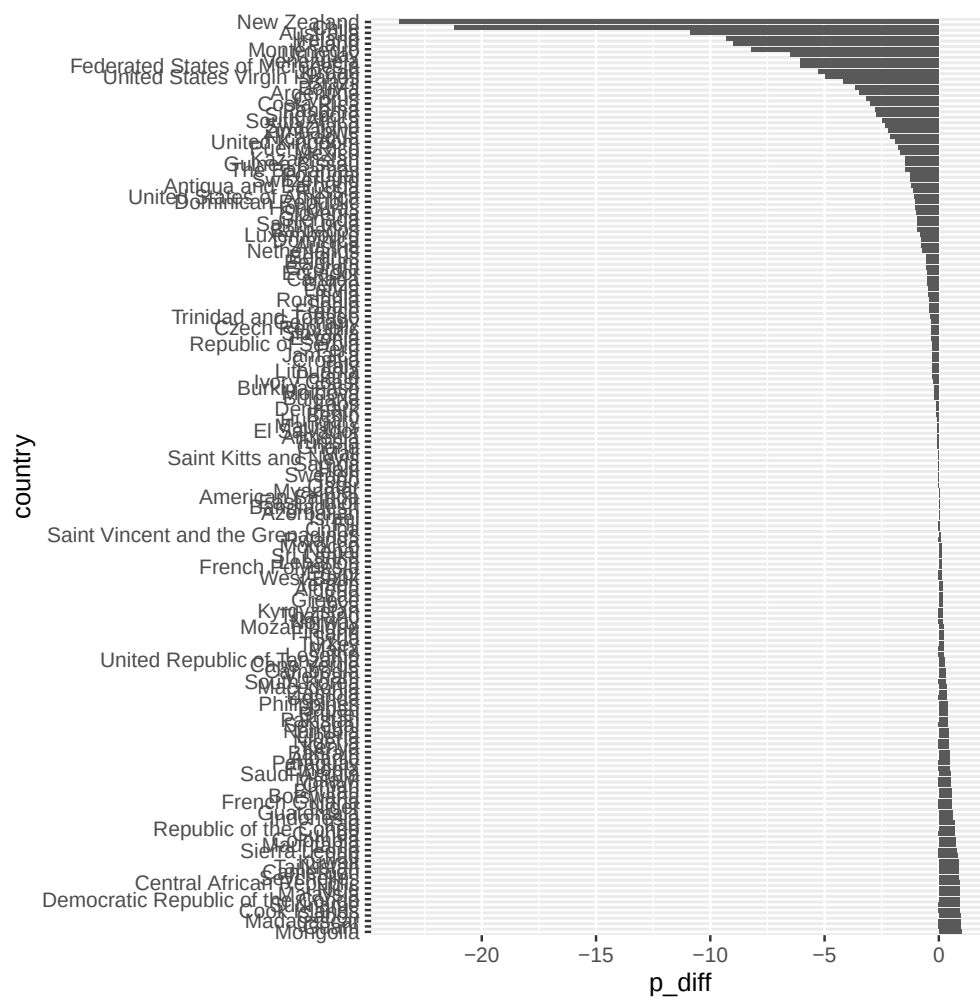
```


14	Federated States of Micronesia	100_200	0.322	0.198	0.124
15	Malta	5_10	0.281	0.183	0.0977
16	Grenada	20_50	0.230	0.134	0.0954
17	Malta	1_2	0.442	0.348	0.0940
18	United States Virgin Islands	5_10	0.241	0.153	0.0880
19	Federated States of Micronesia	5_10	0.141	0.0690	0.0725
20	Cyprus	100_200	0.277	0.207	0.0693
21	Dominica	1000_5000	0.0674	0	0.0674
22	Saint Lucia	1000_5000	0.0940	0.0311	0.0629
23	Niue	20_50	0.342	0.288	0.0545
24	Kuwait	100_200	0.126	0.0718	0.0538
25	Brunei	200_500	0.0788	0.0276	0.0512



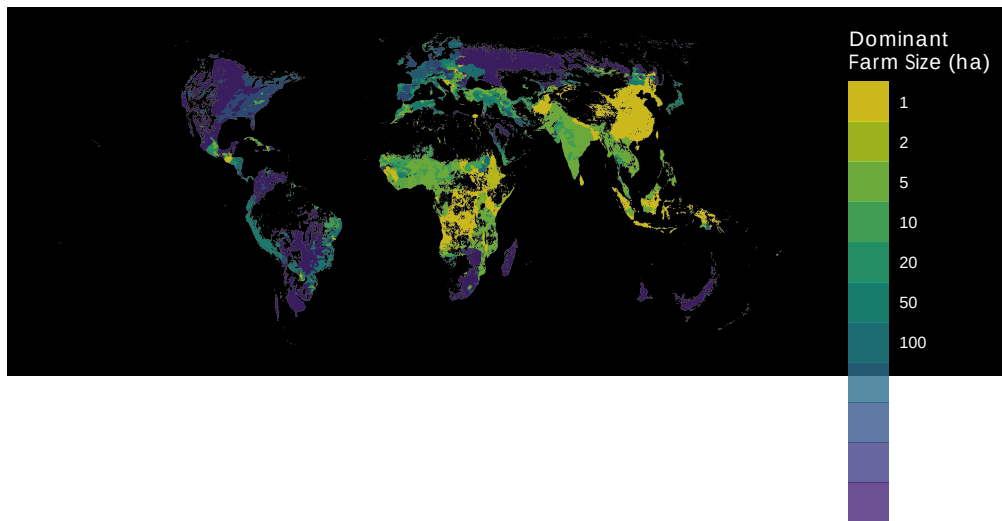
5.4 ESDA Checks

We want to understand which countries/farms sizes are systematically different than the original farm size distribution data



5.5 Final Output

Here is one of the 100 farm size maps generated. We take the mode of all farm size maps and apply a simple spatial smoother.



Session information

```
> sessionInfo()
```

```
R version 4.4.3 (2025-02-28)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Sequoia 15.1.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.0
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/Los_Angeles
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices utils      datasets  methods   base
```

```
other attached packages:
```

```
[1] zoo_1.8-12      viridis_0.6.5    viridisLite_0.4.2 lubridate_1.9.3
[5] forcats_1.0.0   stringr_1.5.1    dplyr_1.1.4       purrr_1.0.2
[9] readr_2.1.5     tidyr_1.3.1      tibble_3.2.1      ggplot2_3.5.1
[13] tidyverse_2.0.0 terra_1.8-42     sf_1.0-20         rworldmap_1.3-8
[17] reshape2_1.4.4 raster_3.6-32    sp_2.2-0          kknns_1.3.1
[21] magrittr_2.0.3  lintr_3.2.0      leaflet_2.2.2     hablar_0.3.2
[25] gmodels_2.19.1  ggthemes_5.1.0   foreach_1.5.2     data.table_1.16.2
[29] renv_1.1.4      formatR_1.14     printr_0.3        knitr_1.49
```

```
loaded via a namespace (and not attached):
```

```
[1] tidyselect_1.2.1 farver_2.1.2     fields_16.3.1    fastmap_1.2.0
[5] lazyeval_0.2.2   rex_1.2.1       digest_0.6.37    dotCall64_1.2
[9] timechange_0.3.0 lifecycle_1.0.4  gdata_3.0.1      compiler_4.4.3
[13] rlang_1.1.4      tools_4.4.3     igraph_2.1.4     utf8_1.2.4
[17] labeling_0.4.3   htmlwidgets_1.6.4 classInt_0.4-11  RColorBrewer_1.1-3
[21] plyr_1.8.9       xml2_1.3.6      KernSmooth_2.23-26 withr_3.0.2
[25] grid_4.4.3       fansi_1.0.6     e1071_1.7-16     colorspace_2.1-1
[29] scales_1.3.0     gtools_3.9.5    iterators_1.0.14 MASS_7.3-64
[33] cli_3.6.3        generics_0.1.3  rstudioapi_0.17.1 tzdb_0.4.0
[37] DBI_1.2.3        proxy_0.4-27    maps_3.4.2.1     vctrs_0.6.5
[41] Matrix_1.7-2     hms_1.1.3       crosstalk_1.2.1  units_0.8-7
[45] glue_1.8.0       spam_2.11-1     codetools_0.2-20 stringi_1.8.4
[49] gtable_0.3.6     munsell_0.5.1   pillar_1.9.0     htmltools_0.5.8.1
[53] R6_2.5.1         evaluate_1.0.1  lattice_0.22-6   backports_1.5.0
[57] class_7.3-23     Rcpp_1.0.14     gridExtra_2.3    xfun_0.49
[61] pkgconfig_2.0.3
```

References

- [1] Z. Mehrabi *et al.*, “The global divide in data-driven farming,” vol. 4, no. November 2020, pp. 154–160, 2020. DOI: 10.1038/s41893-020-00631-0.
- [2] K. Ushey and H. Wickham, *Renv: Project environments*, R package version 1.0.3, 2023. [Online]. Available: <https://CRAN.R-project.org/package=renv>.
- [3] M. Lesiv *et al.*, “Estimating the global distribution of field size using crowdsourcing,” *Global Change Biology*, vol. 25, no. 1, pp. 174–186, 2019. DOI: 10.1111/gcb.14492. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/gcb.14492>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/gcb.14492>.
- [4] A. South, *Rworldmap: Mapping global data*, <https://github.com/AndySouth/rworldmap/>, 2016.